Sustainable Serverless Computing With Cold-Start Optimization and Automatic Workflow Resource Scheduling

Shanxing Pan, Hongyu Zhao, Zinuo Cai, Dongmei Li, Ruhui Ma, *Member, IEEE*, and Haibing Guan

***Abstract*—In recent years, serverless computing has garnered significant attention owing to its high scalability, pay-as-you-go billing model, and efficient resource management provided by cloud service providers. Optimal resource scheduling of serverless computing has become imperative to reduce energy consumption and enable sustainable computing. However, existing serverless platforms encounter two significant challenges: the cold-start problem of containers and the absence of an effective resource allocation strategy for serverless workflows. Existing pre-warm strategies are associated with high computational overhead, while current resource scheduling techniques inadequately account for the intricate structure of serverless workflows. To address these challenges, we present SSC, a pre-warming and automatic resource allocation framework designed explicitly for serverless workflows. We introduce an innovative gradient-based algorithm for pre-warming containers, significantly reducing cold start hit rates. Moreover, leveraging a critical path and priority queue-based algorithm, SSC enables efficient allocation of resources for serverless workflows. In our experimental evaluation, SSC reduces the cold start hit rate by nearly 50% and achieves substantial cost savings of approximately 30%.**

***Index Terms*—Cold start, resource scheduling, serverless computing, serverless workflow.**

I. Introduction

SUSTAINABLE computing in computing science and engineering concerns the consumption of underlying resources to reduce the carbon impact on the environment. Serverless computing [1] is an emerging computing paradigm of sustainable cloud computing. Serverless computing, also called Function as a Service (FaaS), is recognized as the next-generation paradigm of cloud computing and is undergoing extensive research and optimization. It is becoming more and more prevalent in considerable areas, such as web services [2], machine learning inference [3], and entertainment [4]. Multiple major cloud providers, e.g., AWS lambda [5], Google cloud functions [6], IBM cloud functions [7] now offer FaaS as the interface to usage-driven, stateless backend services.

The rise of serverless platforms is not without reason. Compared with the traditional platform as a service (PaaS) [8], FaaS has the following advantages. First, serverless computing offers a lightweight function execution model, providing higher parallelism without the need for manual configuration of containers or virtual machines. Users only need to upload their function code, eliminating resource management complexities associated with traditional PaaS modes. Second, serverless computing adopts a ”pay-as-you-use” billing model, ensuring cost-effectiveness by charging users solely for the resources they consume, without any additional costs for idle resources. Finally, serverless computing eliminates the need for developers to scale resources based on workload fluctuations manually. With its inherent scalability, users can focus on programming without the concern of operation and maintenance.

However, some disadvantages of current serverless platforms limit developers from deploying their applications using the serverless computing paradigm. The cold start [9] of the serverless applications has been a severe challenge during the execution process. A cold start invocation occurs when a function is triggered, but its associated application has not been loaded into memory yet. Compared to warm start invocations, cold starts can lead to resource inefficiencies and prolonged request processing times in serverless functions. Addressing the cold start challenge in serverless computing is currently a subject of active research and development. For example, Jonas et al. [1] alleviate the frequency of cold start by reusing the warm but idle containers in the cluster. Vahidinia et al. [10] use reinforcement learning to pre-warm containers by analyzing function time and invocation frequency. Zhao et al. [11] use the gradient to predict the number of subsequent containers in order to achieve the purpose of pre-warming in advance. However, many of these strategies reduce the cold start hit rate at the expense of computing resources. It has been a critical problem to avoid the cold start with minimum resource consumption.

Moreover, optimizing a workflow is a challenging task, as it involves tackling various complexities. A serverless

Manuscript received 20 February 2023; revised 21 August 2023; accepted 27 August 2023. Date of publication 1 September 2023; date of current version 5 June 2024. This work was supported in part by the Shanghai Key Laboratory of Scalable Computing and Systems, Internet of Things special subject program, China Institute of IoT (Wuxi), Wuxi IoT Innovation Promotion Center under Grant 2022SP-T13-C, and in part by Industry-university-research Cooperation Funding Project from the Eighth Research Institute in China Aerospace Science and Technology Corporation (Shanghai) under Grant USCAST2022-17. Recommended for acceptance by Z. Xu, S. Ren, and O. Rana. *(Corresponding author: Ruhui Ma.)*

Shanxing Pan, Hongyu Zhao, Zinuo Cai, Ruhui Ma, and Haibing Guan are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: bloom66@sjtu.edu.cn; sjtu-zhy@sjtu.edu.cn; kingczn1314@sjtu.edu.cn; ruhuima@sjtu.edu.cn; hbguan@sjtu.edu.cn).

Dongmei Li is with Beijing Institute of Microelectronics Technology, Beijing 100076, China (e-mail: lidm772@163.com).

Digital Object Identifier 10.1109/TSUSC.2023.3311197

2377-3782 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.

workflow generally refers to a batch of functions orchestrated by user-defined logic hosted on serverless platforms. However, scheduling a serverless workflow presents several challenges. To begin with, the orchestration of serverless functions can result in a highly intricate framework, like branch and parallel, which makes it much more challenging to allocate resources for serverless workflows. Moreover, the scheduling of serverless workflows poses an additional challenge due to its inherent complexity. The sheer expanse of the search space for identifying the optimal solution adds to the intricateness of this issue. As a result, designing an algorithm to search for the best configuration automatically is non-trivial.

Therefore, we propose SSC, a pre-warming and automatic resource allocation framework designed for serverless workflows. Our system primarily aims to address the two issues mentioned above in serverless platforms: cold start optimization and resource allocation. We design the Improved Gradient-based Pre-warming Optimization Algorithm to reduce the cold start hit rate while maintaining the high utilization of serverless applications. The strategy of the system depends on the trend of the invocations. When the number of invocations is on the rise, the system adopts an approach of pre-warming corresponding containers according to the direction of the trend. Conversely, while there is a downward trend in invocations, the system will proactively terminate the containers based on the trend. We also develop the Critical Path Optimization Algorithm, which is designed to optimize resource allocation after each execution of a serverless workflow by utilizing information from the workflow's structure and specific execution details. The system analyzes execution results, identifies critical paths, and selects target functions that can reduce resource allocation while maintaining performance. To achieve this goal, the system iteratively evaluates nodes and transitions within the workflow to construct a critical path, which serves as a guideline for resource allocation optimization. Once the critical path is established, the system prioritizes functions based on their impact on cost reduction and uses a priority queue to determine the sequence of function configuration adjustments.

We evaluate the effectiveness of the Improved Gradient-Based Pre-warming Optimization Algorithm in serverless environments through simulations and real-world experiments. We also validate the effectiveness of our system through extensive experiments. To evaluate resource allocation performance, we construct a six-function workflow and use SSC to adjust resource allocation iteratively. We measure each function's runtime, cost, and memory allocation, as well as the overall performance. Our contributions are highlighted as follows:

- We develop SSC, a resource scheduling framework for serverless computing that reduces the waste of computing resources during the course of pre-warming and automates the process of optimizing resource configuration.
- SSC uses the Improved Gradient-Based Pre-warming Algorithm. Our design greatly reduces the cold start hit rate and increases the utilization of serverless applications.
- We design an algorithm to automatically search potential optimized resource allocation strategies while executing the workflow. Our algorithm can adapt to complex workflow structures using the critical path and priority queue, including the branch and parallel.
- We validate the effectiveness of our system with a six-function workflow in serverless environments through simulations and real experiments. SSC reduces the cold start hit rate by nearly 50% and achieves substantial cost savings of approximately 30%.

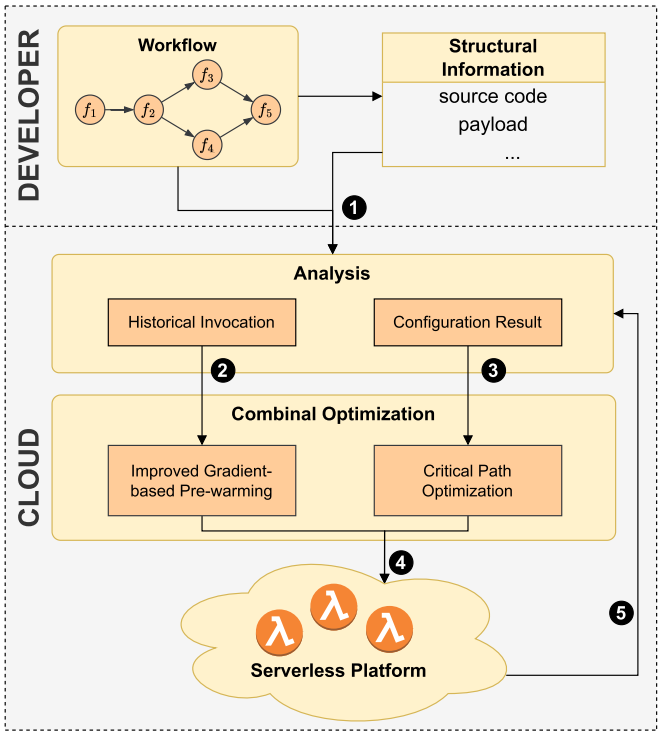


Fig. 1. System overview.

II. System Overview

We design SSC, an automated resource scheduling framework based on serverless computing offering optimized resource configuration while reducing the cold start hit rate with the Improved Gradient-Based Pre-warming Optimization Algorithm. Fig. 1 illustrates the system architecture of SSC. To begin, the user initiates the process by developing the serverless workflow locally. Once the workflow is ready, it is submitted to a chosen serverless platform, such as AWS Lambda or Azure Functions, for execution. The user extracts critical structural information from the serverless workflow, such as the function source code and the function payload. These extracted components play a crucial role in understanding the functionality and requirements of each function within the workflow. The extracted information is then passed on to our frameworks, which leverage this data to further analyze and optimize the execution of the serverless workflow ❶. To reduce the cold start hit rate at the expense of low computing resource waste, we use our Improved Gradient-Based Pre-warming Optimization Algorithm to help the controller

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.

make further decisions. Before each execution of the workflow, our system SSC analyzes function invocations from the dataset of the Historical Invocation to gather insights and information. Based on the slope of the function invocation rate, our system SSC performs pre-warming operations when the slope is greater than zero, ensuring that containers are prepared in advance to handle increasing workload demands. Conversely, when the slope is less than zero, our system performs termination operations to efficiently manage resources and avoid unnecessary container instances ❷.

After each execution of the serverless workflow, SSC takes further action to optimize the resource configuration. It utilizes the information gathered from the workflow's structure and the specific execution details from the dataset of the Configuration Result to make informed adjustments. This involves analyzing the results of the execution, identifying critical paths, and selecting target functions that can reduce resource allocation while maintaining performance. To achieve this, SSC constructs a critical path by iteratively evaluating the nodes and transitions within the workflow. This critical path serves as a guideline for resource allocation optimization. Once the critical path is established, SSC prioritizes functions based on their impact on cost reduction. It utilizes a priority queue to determine the order of function configuration adjustments ❸. By iteratively adjusting the resource configuration using these algorithms, SSC effectively fine-tunes the serverless functions to achieve the optimal balance between cost and performance. These adjustments are automatically triggered each time the serverless workflow is executed on the serverless platform, relieving the user from the burden of manually managing the resource configuration ❹. The result executed by the serverless platform will be sent back to the analysis system to enrich the dataset of the Configuration Result. The content of the Configuration Result will facilitate the more in-depth resource configuration optimization during subsequent execution ❺.

Algorithm 1: Improved Gradient-Based Pre-warming Optimization Algorithm.

Input: numbers of serverless function invocations $Invocation$, numbers of the existing alive containers $AliveContainer$
Output: numbers of containers to pre-warm or terminate $HandleContainer$

Function `Improved Gradient-based Pre-warming(`$workflow, AliveContainer$`)`**:**
  $Slope \leftarrow SlopeModel(Invocation)$
  if $Slope > 0$ **then**
    /* rising cases */
    $PredictedContainer \leftarrow Invocation + Slope * WindowSize$
    if $PredictedContainer > AliveContainer$ **then**
      /* pre-warm the containers */
      $HandleContainer \leftarrow PredictedContainer - AliveContainer$
    end
    else
      /* no change */
      $HandleContainer = 0$
    end
  end
  else
    /* falling cases */
    $PredictedContainer \leftarrow Invocation - Slope * WindowSize$
    /* terminate the containers */
    $HandleContainer \leftarrow AliveContainer - PredictedContainer$
  end
  return $HandleContainer$
End Function

III. Mitigating Cold Start in FaaS

A. Preliminary

We use our insights from our characterization to design an algorithm to avoid the cold start. The goal is to reduce the number of cold start invocations with the minimum resource waste. To better learn about the cold start, we reference the two parameters proposed by Shahrad et al. [12] to describe how cold start happens and the importance of these parameters.

- *Pre-warming time:* The Pre-warming time refers to the interval since the last execution before it loads the function image expecting the following invocation.
- *Keep-alive time:* The Keep-alive means the time during which a function's image is kept alive after it has been loaded to memory.

Fig. 2 shows how the Pre-warming time and Keep-alive time affect the warm and cold start. If the functions are executed after their image is loaded to memory, the serverless applications can get warm start invocations. In Fig. 2, an invocation that happens before Keep-alive time ends and that happens as the

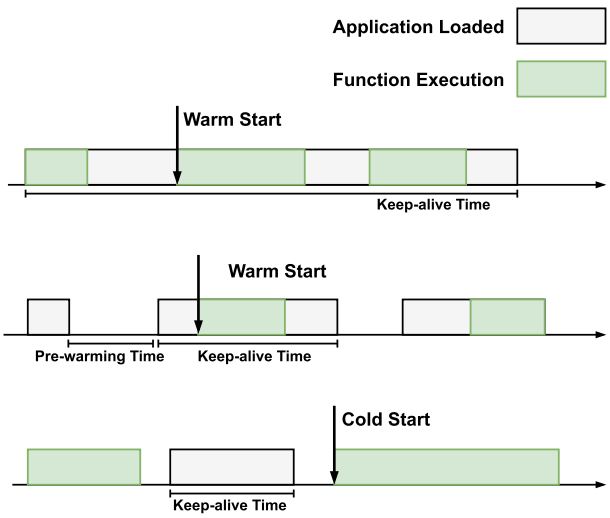


Fig. 2. Timelines to illustrare pre-warming and keep-alive time affect function start types.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.

application is reloaded after the Pre-warming time is a warm start. Accordingly, we can control the Keep-alive time and Pre-warming time to dodge the cold start. However, from the top and middle timelines, both ways can keep the applications getting a warm start. While the Keep-alive time of the top timeline is much longer than that of the middle one, it is fluent to keep the serverless applications alive at the operational level. But during the Pre-warming time, the serverless applications are in idle time. This action can bring a lot of resource waste to the applications. We take this problem into consideration and come up with an adaptive pre-warming method. Our algorithm can pre-warm the serverless applications according to the potential increase of invocations of the serverless functions. Also, we terminate idle serverless applications in advance to escape from resource waste.

B. Improved Gradient-Based Pre-Warming Optimization Algorithm

The cold start has a severe side effect on the performance of serverless functions, mainly latency-sensitive functions. The cold start causes delays as the container needs to be initialized, resulting in longer response times for the initial function invocation. The common method to reduce the cold start hit rate is pre-warming the containers in a fixed time. Nevertheless, this approach does not significantly improve performance when the invocations for serverless functions dynamically fluctuate. This approach can effectively reduce the cold start hit rate but at the cost of a relatively lower container utilization rate. To address this problem, we develop the Improved Gradient-Based Pre-warming Optimization Algorithm.

Algorithm 1 describes the primary process of the Improved Gradient-based Pre-warming Optimization Algorithm. Our algorithm aims to reduce the number of cold start invocations with minimum resource waste. The system records the number of invocations and uses these numbers to describe the trend of the invocations. We set a window size to represent the frequency of the count of invocations. We use 5 min as a standard to represent window size [11], which can achieve a better performance provided that it will not bring too much tracking overhead to the system. The number of the invocations recorded will be sent to the slope model, which can differentiate between the rising trend and the falling trend to calculate the potential invocations of the serverless functions. The system takes different measures on the two situations we separated: rising cases and falling cases.

Rising Cases: The slope model marks the situation when the numbers of invocations have an apparent trend to soar as rising cases. In this case, the containers will be automatically pre-warmed because the rising invocations can easily lead to the circumstance where there aren't enough containers to handle the invocations of the serverless functions, and the cold start hit rate will skyrocket. So the system pre-warms the containers due to the results from the slope model. However, some serverless platforms, such as OpenWhisk, usually sets their containers' alive time much longer than the execution time of the serverless functions. If our system keeps pre-warming the containers, it can produce a deadly vast waste of resources. To prevent such a situation from happening, our system tends to compare the numbers of the existing alive containers with the predicted invocations. The containers will be pre-warmed, given the numbers of the existing alive containers that are lower than the predicted invocations.

Falling Cases: The long alive time of the containers can also cause problems with falling cases. When the invocations of the serverless functions begin to descend, too many alive containers seem to be no need. The existence of the containers may consume resources but don't complete any computing jobs, which causes a waste of resources. Consequently, our algorithm tries to terminate the containers in the falling cases in advance. Due to the existence of a lifecycle for containers, when the container exceeds its lifecycle, it will be automatically terminated. To minimize the impact of terminated containers on subsequent container calculations, our algorithm chooses to terminate the initially created containers which are not in use first since they have relatively shorter lifetimes. We refer to the container list as a priority queue, in which it's better for us to manage the survival situation of the containers. We initialize the queue with an empty list. Then the container will enqueue as long as being created, and the order of the queue is determined by the time the container is created. When falling cases occur, our algorithm terminates the containers due to the queue, which represents dequeuing in the priority queue. We filter the containers based on the order in the queue, starting from the head of the queue. When a container is in the running state, we will continue filtering toward the end of the queue. Once we find an idle container, we will terminate it, remove its information from the container list, and shift the subsequent containers forward. This process will be repeated according to the number of containers calculated from the Improved Gradient-based Pre-warming Optimization Algorithm.

IV. Cost Optimization Algorithm Within Performance Constraint

A. Preliminary

In order to schedule resources for the serverless workflow in an efficient manner, we first formulate a model of the serverless workflow, which we use to operate all the configurations. Next, we introduce an algorithm that iteratively selects a function in the workflow and reduces the resource allocation of that particular function while attempting to maintain the overall performance.

Considering the branch and the parallel, the serverless workflow of a serverless application is defined as a weighted DAG G:

$$G = (V, E, RT, TP, MEM, C) \tag{1}$$

where:

- V represents the vertices in graph G.
- $E \subset V \times V$ represents the edges in graph G.
- $RT : V \to [0, +\infty)$ is a response time function, indicating the response time of each serverless function.
- $TP : E \to (0, 1]$ is a transition possibility function, indicating the possibility of the occurrence of the edge.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.

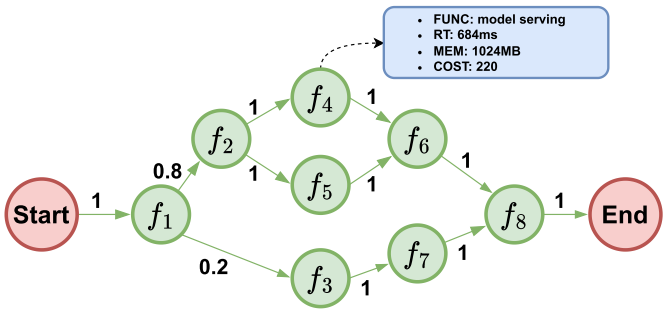


Fig. 3. Sample workflow.

- $MEM : V \to \mathbb{N}$ is a memory function, indicating the memory allocation of each serverless function.
- $C : V \to (0, +\infty)$ is the cost function, indicating the cost per invocation of each serverless function

We give an example of modeling in Fig. 3. We use green nodes to represent the serverless functions in the workflow. In addition, there are two red nodes in Fig. 3 representing structural nodes containing no serverless functions. The edges between the nodes are the function dependencies, indicating the order of the execution of functions. And the numbers above the edges are the transition probability TP, which indicates the direction of the execution of functions. For example, the TP is 0.8 from f_1 to f_2 and 0.2 from f_1 to f_3, which indicates that after finishing executing f_1, the workflow will choose to execute f_2 in 80% of cases and execute f_3 in 20% of cases. Moreover, each node contains specific details regarding function execution. We show this in the square connected to node 4. There is the name of the serverless function this node represents, the response time RT, memory allocated to this node MEM, and the cost of this node C.

B. Critical Path Optimization Algorithm

Our goal is to minimize costs while maintaining performance. To achieve this, we need to identify a function that can reduce resource allocation without significantly impacting overall performance. We develop an algorithm called Critical Path Selection Algorithm, which selects the critical path in the serverless workflow. Then, we design another algorithm called Critical Path Resource Configuration, which directly selects the target function from the critical path according to the priority of each function. In Algorithm 2, the Critical Path Selection Algorithm first appends the start node of the serverless workflow into the critical path $cPath$ and then goes into an infinite loop until it hits the end node. In the loop, the Critical Path Selection Algorithm gets the head of the critical path $cPath$ and puts it into a variable $currNode$. In the case of branches, we take into account each scenario. However, our objective is to schedule the branch that is invoked most frequently first. This approach allows us to minimize costs rapidly and efficiently. Regarding parallel, we prioritize scheduling the branch with the longest execution time. This decision is based on the understanding that it has a more significant impact on overall performance. Subsequently, we schedule the remaining branches in order to further reduce costs.

Algorithm 2: Critical Path Selection Algorithm.

```
Input: serverless workflow workflow
Output: critical path cPath
Function Critical Path Selection(workflow):
    cPath.append(workflow.startNode)
    while True do
        currNode ← cPath.head
        if currNode is workflow.endNode then
            /* until hit the end node */
            break
        end
        if len(currNode.successors) == 1 then
            /* no branch or parallel */
            cPath.append(currNode.successors[0])
        end
        else if TP(currNode, node) == 1 for
         node ∈ currNode.successors then
            /* parallel */
            target ← path with maximum runtime
            merge(cPath, target)
        end
        else
            /* branch */
            target ← path executed last time
            merge(cPath, target)
        end
    end
    return cPath
End Function
```

If the number of successors of the $currNode$ equals 1, which means there is no branch or parallel, the Critical Path Selection Algorithm fetches the successor of the $currNode$ and puts it into the $cPath$ and starts another loop. Else, if all the transition possibilities from the $currNode$ to its successors equal 1, which means all the branches will be executed in parallel, the Critical Path Selection Algorithm searches all the branches and finds the one with the least response time and puts it into the $cPath$. Else, the structure of this part of the workflow must be a branch. The Critical Path Selection Algorithm finds the branch executed last time and puts it into the $cPath$, such that after several configurations, the weight of each branch resource allocation is equal to their possibility of occurrence.

After that, in Algorithm 3, the Critical Path Resource Configuration first executes the Critical Path Selection Algorithm to get the critical path $cPath$. Then, the Critical Path Resource Configuration initializes a priority queue pq, where the Critical Path Resource Configuration gets all the nodes in $cPath$, gives them the highest priority, and puts them into pq. Next, the Critical Path Resource Configuration goes into an infinite loop until the highest priority in the pq equals zero. In the loop, the Critical Path Resource Configuration fetches a function with the highest priority, takes one step of memory off this function, and regenerates the priority. Because we want to reduce the cost while trying to maintain the response time the same, we set the

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: Critical Path Resource Configuration.

Input: serverless workflow $workflow$
Output: resource configuration $config$
Function `Critical Path Resource Configuration(`$workflow$`):`
  $cPath \leftarrow CriticalPathSelection(workflow)$
  foreach $node \in cPath$ **do**
    $pq.insert(node, node.priority)$
  end
  while *True* **do**
    $currNode \leftarrow pq.pop()$
    if $currNode.priority == 0$ **then**
      `/* until the highest priority is zero */`
      break
    end
    $currNode.mem \leftarrow currNode.mem - mem_b$
    execute $node$ to get the decrease of the cost Δc, and the increase of the response time Δrt
    `/* priority equals reduced response time divided by the increased cost plus 1 */`
    $priority \leftarrow \mathbf{max}(\frac{\Delta cost}{\mathbf{max}(\Delta rt,0)+1}, 0)$
    $pq.insert(node, priority)$
  end
  return $config$
End Function

priority of each node to the reduced cost divided by the increased response time plus 1. Hypothetically, a function with the most reduced cost time in one-step allocation is more likely to have the most reduced cost in the next-step allocation. By setting the priority this way, we guarantee that the performance constraint is met approximately at the lowest cost. After the loop breaks, the Critical Path Resource Configuration returns the configuration scheme.

Case Study: Here we take the serverless workflow in Fig. 3 as an example to show how the SSC works to optimize the cost under the performance constraint. Instead of illustrating the index of the nodes, we show the response time of each serverless function. As mentioned earlier, this workflow contains a branch and a parallel, and we demonstrate them with different colors. Fig. 4 shows the process of configuration. The first graph in Fig. 4 shows the initial workflow, as we have shown before. The second graph shows a certain execution of this serverless workflow, where we use grey to represent the nodes that have not been executed. Concretely, this workflow chooses to execute f_2 after the execution of f_1. The SSC then runs the Critical Path Selection Algorithm to select the critical path from this workflow. In the branch, as we executed f_2 before, the Critical Path Selection Algorithm will take the branch with the f_2 into the critical path. Similarly, in the parallel, the Critical Path Selection Algorithm chooses the branch with less response time, which is f_5. We show the selected critical path in the third graph of Fig. 4 in yellow. After that, the SSC runs Critical Path Resource Configuration, trying to reduce cost. The Critical Path Resource Configuration puts the critical path into a priority queue using the priority of the nodes and takes one node with the highest priority from the queue. The Critical Path Resource Configuration then decreases the memory allocation of this node and regenerates its response time, cost, and priority. In this case, the Critical Path Resource Configuration selects the f_5, and after the configuration, the response time of f_5 increases to 256 ms. We can see that we decrease the overall memory allocation without increasing the overall response time. If the reduced cost of the f_5 is high, it's more likely we will continue to decrease the memory allocation of the f_5 the next time.

V. Evaluation

A. Experiment Setup

Hardware Environment: Our experiment is deployed on the server running on Linux, which has 24 CPUs and 128 MB of memory.

Software Environment: We use Docker version 18.09.7 in our experiment. We use the Kind to build a Kubernetes cluster and deploy OpenWhisk platform version 1.0. We define a PyTorch version 1.6 image called runtime as the worker's environment.

Objectives: We will illustrate the effectiveness by evaluating SSC in the following sections. In Section V-B, we use the simulator and the real serverless platform to evaluate the performance of our Improved Gradient-Based Pre-warming Algorithm, which aims to reduce the cold start invocations with minimal overheads. In Section V-C, we construct an experimental workflow using six different serverless functions. Then we use SSC to configure this workflow and trace the response time, cost, and memory allocation of each function, as well as the overall performance of the workflow.

B. Improved Gradient-Based Pre-Warming

1) Experiment Configurations: Simulator: Evaluating our optimization algorithm requires a long execution time to assess applications with infrequent invocations and a large space of configurations. To confine our experiment time, we use simulations. We build a simulation that allows us to compare our optimization with the real invocations.

To simulate our experiment deployed on real serverless applications, we generate a class to represent the real container, the class holds many key characteristics such as the alive time of the container, the status which is made up of IDLE, RUNNING, and TERMINATED, and the status transformation. We also create a list to record the containers in the RUNNING state. We mark the cold start container with Arabic Numerals while the warm start container with the English Alphabet. We simulate the function execution time equal to the average actual serverless function execution time. To simulate the function execution, the simulator will trace the status of each container and search for the IDLE container whose state will be turned into RUNNING. After ending the simulated function time, the state will be back to IDLE, loop these steps, and finally be transformed into TERMINATED when ending the alive time.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.

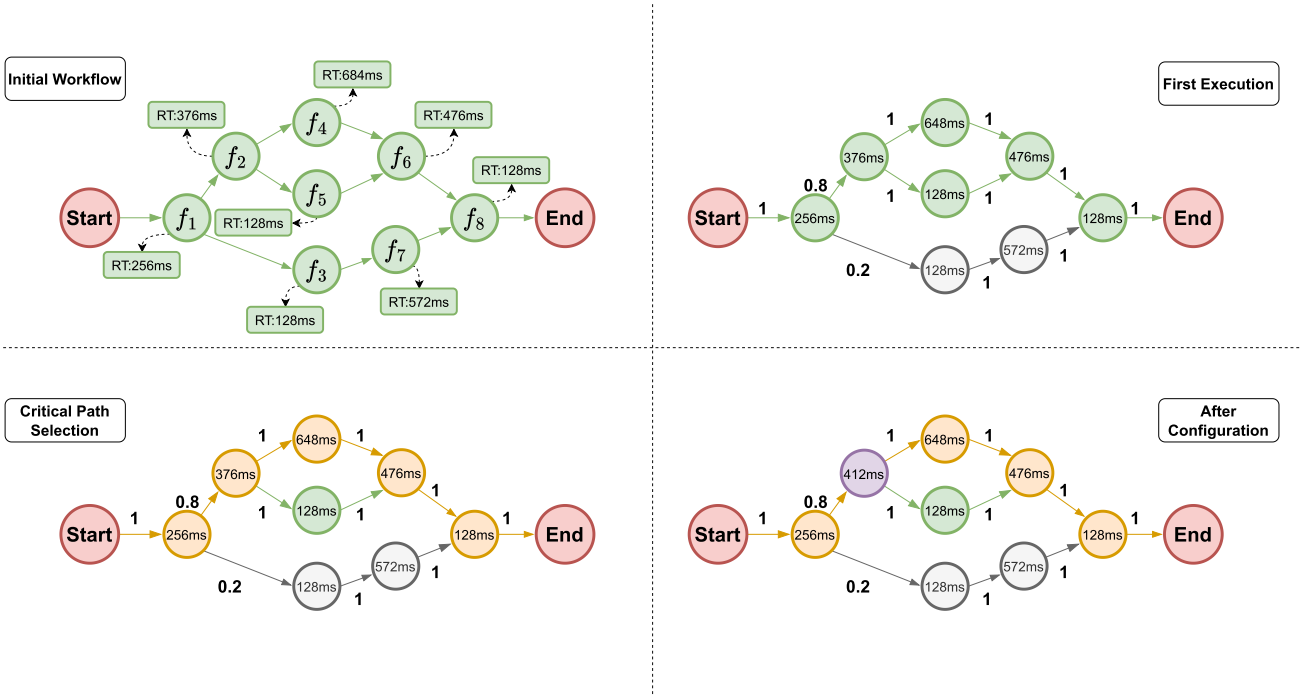


Fig. 4. Configuration process. The initial workflow is a workflow with a branch and a parallel. In the first execution, the Critical Path Selection Algorithm is employed to identify the critical path, taking into account function response times. Subsequently, the Critical Path Resource Configuration is used to decrease costs. This involves placing the critical path in a priority queue, adjusting resource allocation, and updating function details. Finally, the optimization of the workflow is concluded.

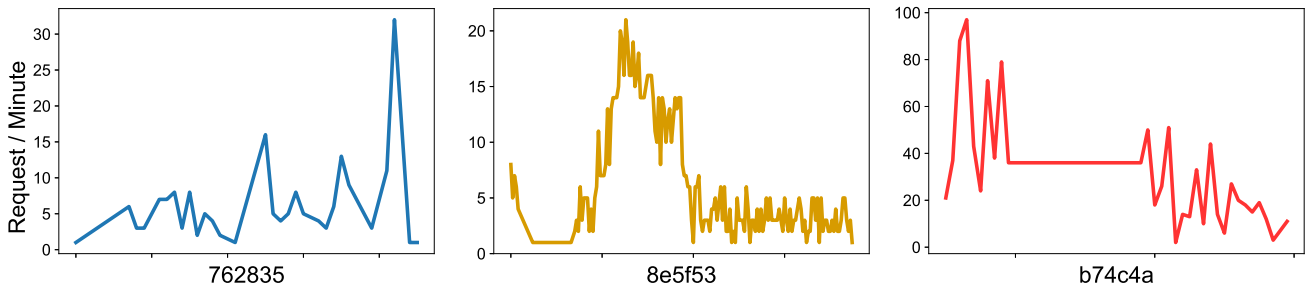


Fig. 5. Traces. The traces come from the records of the function invocations in Microsoft's Azure Functions [13] for two weeks starting on 2021-01-31. We choose three traces to monitor the actual function invocations. Each sub-figure represents one kind of function. The x-axis donates the time period, and the y-axis donates the number of function invocations per minute. 762835 and 8e5f53's front half refer to rising cases, while b74c4a and 8e5f53's back half refer to falling cases.

Real Experiment: To show our optimization can be easily implemented in a real system with minimal overheads, we use our OpenWhisk implementation. We run our experiment on a server running Linux. The server has 24 CPUs and 128 MB of memory. We use the Kind to build a Kubernetes cluster and deploy OpenWhisk platform version 1.0 to trail our Improved Gradient-Based Pre-Warming.

Workloads: In Fig. 5, we reference the traces of functions invocations in Microsoft's Azure Functions [13] for two weeks starting on 2021-01-31. We choose four traces from the data to evaluate the feasibility of our Improved Gradient-Based Pre-warming Optimization Algorithm. The first two traces represent the rising cases, in which the number of requests tends to soar, judged by our slope model, while others stand for the falling cases.

2) Simulation Results: Rising Cases. In Section III, we mention that we refer to the rising cases as the situation in which the requests received by the serverless application have an upward tendency. Through the experiment, we find that such cases always have a relatively high cold start hit rate. We blame this scene on the skyrocketing requests which make serverless applications always in demand. Our algorithm aims to predict the potential increase of the requests from the serverless functions and offset the inadequate part compared to the existing serverless applications by pre-warming them. In Fig. 6, we can find that the cold start hit rate of the rising cases without any measures

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.

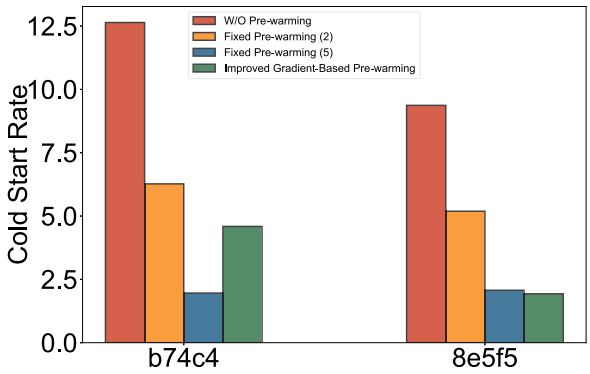


Fig. 6. Cold start hit rate of rising cases.

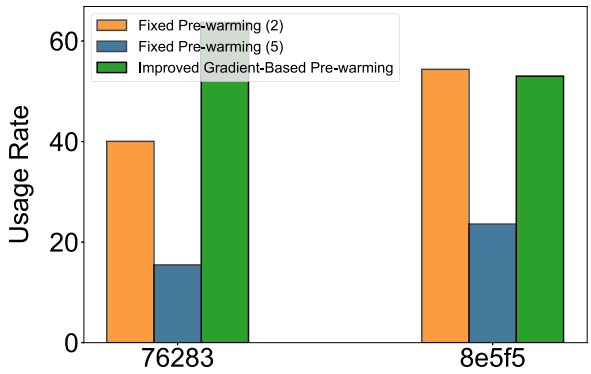


Fig. 7. Containers' utilization rate of rising cases.

to pre-warm the dockers is separately 12.74% and 11.31% on average. But with the aid of our Improved Gradient-Based Pre-warming Optimization Algorithm, the corresponding cold start hit rate can be optimized to 4.58% and 1.74%. The Improved Gradient-Based Optimization Algorithm has a positive effect on handling dynamically changing requests.

Falling Cases: In Section III, the concept of the falling cases is explained as the situation where the requests of the serverless function become less and less. We acknowledge that the falling cases may cause a waste of computing resources because of the long alive time of the docker when the digit of the requests from the serverless functions may be far less than the existing serverless applications. This phenomenon can cause idle time for serverless applications, which can be saved for other use. As a result, our algorithm can terminate given serverless applications according to a sliding trend. But it will not affect the initial cold start hit rate a lot. In Fig. 8, we figure out that the initial cold start hit rate of the falling cases is 5.73% and 4.77%. But after we optimize the utilization rate of the dockers, the cold start hit rate remains nearly date against the initial cold start hit rate. Based on the low cold start hit rate, the utilization rate of the dockers comes up to 58.83% and 63.02% separately.

Comparison With Traditional Method: The traditional way to avoid a cold start is to pre-warm quantitative containers in a fixed time. We will compare the traditional method with our Improved Gradient-Based Pre-warming Algorithm. The core of the traditional method is the number of dockers pre-warmed, so we choose two numbers for the comparison (pre-warm 2/5 dockers every 5 minutes). In Figs. 6 and 8, we found that the cold start hit rate of the fixed pre-warming (5) is lower than that of the fixed pre-warming (2), bringing the expense that the fixed pre-warming (5) has relatively low usage of the pre-warmed dockers. This action can generate huge overhead for applications. However, our Improved Gradient-Based Optimization Algorithm can avert such a story. As shown in Figs. 6 and 8, the algorithm can keep a perfect balance between the cold start hit rate and the dockers' utilization rate. We can conclude that the Improved Gradient-Based Pre-warming Algorithm is better adapted to handle dynamically changing requests.

3) Real Experiment Result: We choose two traces from Microsoft's Azure Functions [13], which represent the actual function invocations for two weeks starting on 2021-01-31 on our

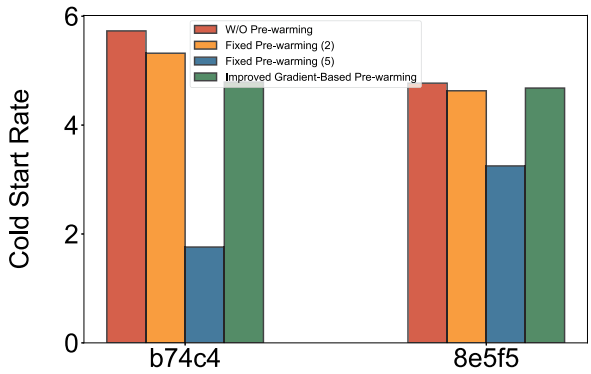


Fig. 8. Cold start hit rate of falling cases.

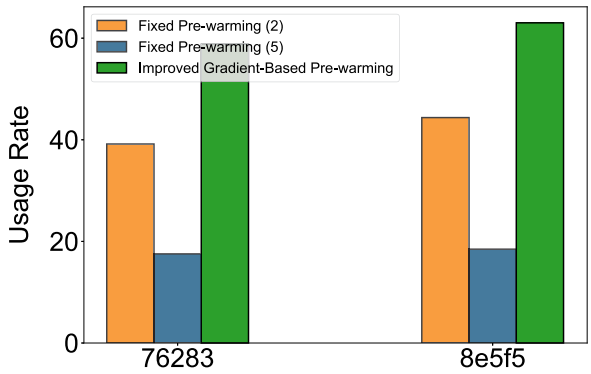


Fig. 9. Containers' utilization rate of falling cases.

OpenWhisk deployment: one represents the rising cases, and the other stands for the falling cases. We compare the cold start hit rate between the policy with OpenWhisk's default policy and the policy with the Improved Gradient-Based Pre-warming Algorithm.

The role of the proxy is very important in the process of the Improved Gradient-Based Pre-warming Algorithm in the serverless platform. It requests the invoker to pre-warm and terminate the containers. In Table I, we illustrate the effectiveness of our Improved Gradient-Based Pre-warming Algorithm. We elect one rising case (func:76283) and one falling case (func:8e5f5) as the workload of our experiment. We find that the cold start hit rate

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE COMPARISON OF THE COLD START HIT RATE BETWEEN OPENWHISK'S DEFAULT POLICY WITH IMPROVED GRADIENT-BASED PRE-WARMING ALGORITHM

	OpenWhisk	Improved Gradient-Based
76283	14.57%	5.82%
8e5f53	7.41%	4.36%

TABLE II
SERVERLESS APPLICATION

Application	Memory (MB)	Multi-CPU
Matrix Multiplication	24	✔
Chameleon	465	✗
Float Operation	4.3	✗
Linpack	42	✔
Model Serving	290	✔
Pyaes	4.6	✗

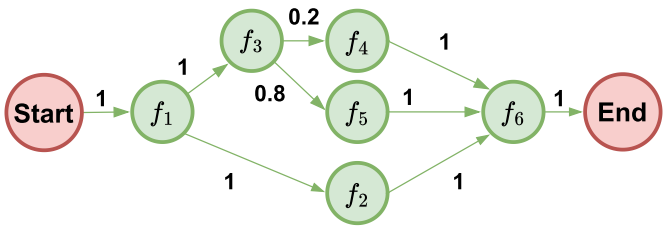


Fig. 10. Experimental workflow.

of the rising case is higher than that of the falling case. However, our pre-warming policy can shake approximately 8.8% cold start hit rate in the rising case and almost 3.05% in the falling case. Although the performance of the Improved Gradient-Based Pre-warming Algorithm is better in the rising cases, the policy can save more computing resources in the falling case. In our experiment, we find that the utilization rate of the pre-warmed containers is more considerable in the falling case. In the falling case, we find that we save about 11% utilization compared to OpenWhisk's default policy.

C. Cost Optimization Algorithm Under Performance Constraint

Experiment Setup: To verify the effectiveness of SSC, we construct a serverless workflow consisting of six serverless functions, including CPU-intensive and memory-intensive functions selected from Table II, shown in Fig. 10. As we can see, this sample workflow contains a parallel and a branch nested in the parallel. We start by allocating each serverless function 768 MB of memory and convert this serverless workflow into a weighted DAG as we described in Section IV. Then, we put the weight DAG into the SSC and continuously iterate until we get the lowest cost to satisfy the user-defined SLO (Service Level Objective). In the process, we record the changes in runtime and costs of each serverless function, as well as the overall runtime and cost, shown in Figs. 11 and 12. We record the runtime and costs of each serverless function to observe the relationship between costs and function characteristics to support the validity of our approach. Additionally, we record the overall runtime and cost to demonstrate the effectiveness of our solution. Also, we record the memory allocation of each function, illustrated in Fig. 13.

Memory Allocation: As we can see from Fig. 13, There are four functions whose memory allocation drops a lot, namely, f_1, f_2, f_5, and f_6, while the other two functions' memory allocation remains almost the same. That's because, in this workflow structure, the f_1, f_2, and f_5 are the performance bottleneck, while f_6 is a necessary node. Meanwhile, f_3 and f_4 are less critical, and changing their resource allocation cannot affect the overall performance.

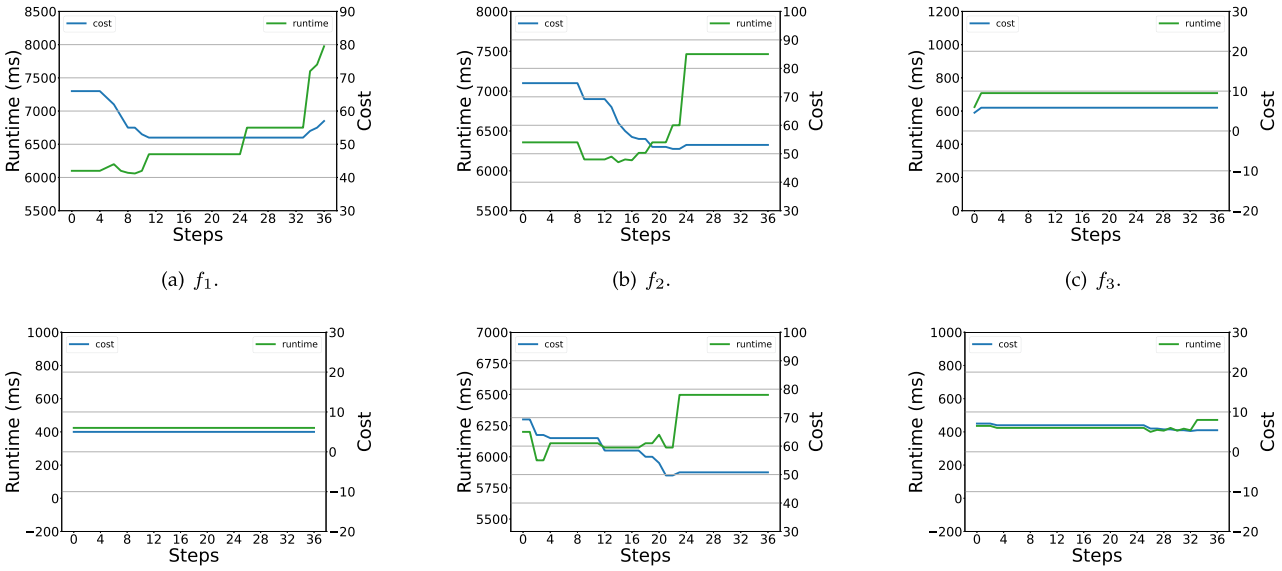
The Runtime and Cost of Each Function: In Fig. 11, we show the runtime and cost of each function. As we mentioned before, f_1, f_2, and f_5 are the performance bottleneck of the serverless workflow. We can see that the runtime of each function rises at first and then maintains the same level, while the cost of each function constantly drops during the process. As mentioned in Section IV, the SSC iteratively puts the selected critical path into a priority queue and chooses the one with the highest priority to decrease its resource allocation. As a result, the SSC can find the trade-off between the performance and cost of each function and the whole workflow.

The Overall Performance: Fig. 12 shows the changes in the overall runtime and cost. As we can see, the overall coat constantly drops, while the overall runtime increases until it reaches the user-defined SLO. Because of the branch in the workflow structure, the cost of the workflow fluctuates violently. But as the f_2 is a performance bottleneck, the runtime remains at the same level. After the configuration phase ends, we execute the workflow several times using the same configuration. As shown in Fig. 12, the runtime is pretty stable, indicating that our configuration can fully satisfy the user's demands.

VI. RELATED WORK

Serverless Computing: With the advancement of virtualization technology, cloud computing is shifting towards a more abstract and lightweight direction. As a consequence, serverless computing—also known as function-as-a-service (FaaS)—has emerged as a highly promising area of research, drawing significant attention from both academic and industrial communities. It is important to note that serverless computing does not imply the absence of servers in a cloud computing system. Rather, it involves a shift in responsibility for server management and maintenance from cloud computing customers to cloud computing providers. As a result, customers can focus solely on code logic and programming, without the burden of managing the underlying infrastructure. Compared with traditional cloud computing, serverless computing provides a more cost-effective choice of cloud service use because of its scalability and "pay-as-you-use" billing mode.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.



(a) f_1. (b) f_2. (c) f_3.

(d) f_4. (e) f_5. (f) f_6.

Fig. 11. Runtime and cost of each function. (a) The Changes in Runtime and Costs of f_1. (b) The Changes in Runtime and Costs of f_2. (c) The Changes in Runtime and Costs of f_3. (d) The Changes in Runtime and Costs of f_4. (e) The Changes in Runtime and Costs of f_5. (f) The Changes in Runtime and Costs of f_6.

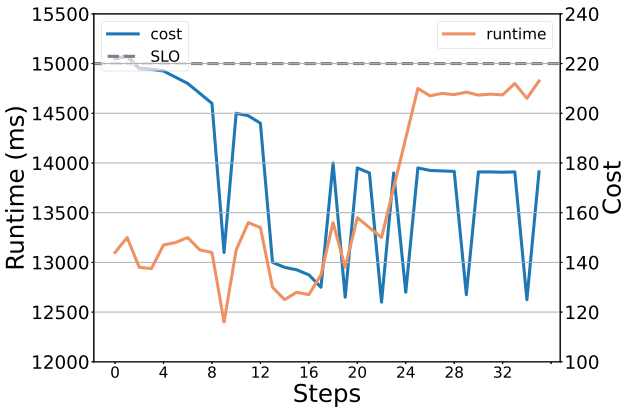


Fig. 12. Overall performance trace.

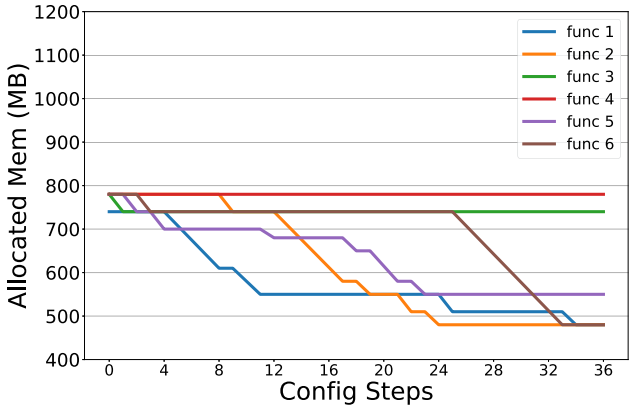


Fig. 13. Memory allocation of each function.

Serverless computing has widespread applications in all walks of life. For Internet of Things (IoT) [14], [15], bringing stateless functions closer to mobile devices can not only exploit the advantages of serverless computing, but reduce the energy consumption and communication latency. Serverless computing has been employed in machine learning training [16], [17], [18], [19] to provide high parallelism and model serving to handle high concurrency requirements. For model training, Jiang et al. [18] provide a comprehensive performance analysis of training deep models atop serverless platforms, and Siren [19] employs reinforcement learning for resource allocation to optimize the economic cost and resource utilization. Existing works consider model partition [20], batch processing [21], cold start optimization [22], and other techniques [23] to improve the efficiency of serverless model serving.

Resource Allocation. Enabling increased parallelism without requiring manual configuration of containers or virtual machines is one of the defining characteristics of serverless computing, with the advantage of removing the configuration burden from the user. However, it also comes with significant disadvantages. For example, the flexibility and high scalability of the serverless workflow make it hard to predict the scale of the next execution. So in most cases, there are configurations that achieve better performance, better cost, or both, than the ones from current offerings. To address these issues, extensive research has been conducted from various aspects.

One area of focus is resource configuration optimization, where Bilal et al. [24] propose decoupling memory and other resources for serverless applicant functions and design an automation resource configuration policy to reduce the execution time or save the costs for serverless customers. Decoupling memory and CPU resource allocations [25] leads to a better

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.

combination of performance and cost of the resource allocation for serverless functions. Natural dynamic pricing scheme [26] and feedback mechanisms allow optimal resource allocation. The author applies reinforcement learning [27] to the resource allocation of serverless computing and generates an effective scaling policy, which is more efficient than the default automatic scaling policy.

Another aspect of resource allocation is performance optimization. Freshen [28] which is invoked by the serverless infrastructure proactively allows developers or providers to establish or warm connections, proactively fetch data or perform actions to reduce overheads when the serverless function runs. KneeScale [29], designed to dynamically adjust the number of function instances, uses the Kneedle algorithm to efficiently scales serverless functions under resource budget constraints in edge systems. GWO-RIL [30] which focuses on intra-datacenter network path parallelizing the jobs allocated to the event queue and the dispatcher of the serverless framework. A load balancer [13] for FaaS platforms which place functions in the VM according to the availability of harvested resources to reduce resource contention while maintaining the function cold start rate low. A performance characteristic curve (PCC) [31] captures the relationship between resources and performance compactly for optimal resource allocation. In comparison, our approach primarily focuses on improving the resource allocation algorithm, enabling better resource allocation performance for workflows with complex structures and in vast resource spaces.

Cold Start: Serverless computing functions encounter a significant challenge known as the cold start problem [32], [33] during the execution process. When the scheduler receives an inference request from the user, it verifies if there is an initialized and idle runtime available. In the absence of a suitable function runtime, the system must initialize a new runtime to fulfill the request, resulting in a cold start. Cold starts entail additional time, including function scheduling, software environment initialization, and application-specific function execution [1]. Many solutions have been proposed to address the cold start problem.

One approach is to perform container pre-warming. Pagurus [1] alleviates the frequency of cold start by reusing the warm but idle containers in the cluster. Reinforcement Learning [10] uses RL to pre-warm containers by analyzing the function time and the frequency of the function invocation. Prebaking [32] uses a prebaking technique that restores snapshots of previously created functions processes to reduce the time of cold start. Shared pool [34] maintains a user-shared pool of warm pods that will stand by and be immediately available to functions with increasing demand.

Additionally, application-level and system-level optimizations have also been proposed to tackle the cold start problem. Defuse [35] uses patterns to construct a function dependency graph by mining strong or weak dependencies in function invocation, to make the connected components scheduled and then diminish the occurrences of cold starts. FaaSLight [36] is an application-level approach to optimizing the cold-start latency of serverless functions by reducing the size of executed code. Function Fusion [37] mitigates the cold start by fusing a workflow with a possible parallel execution of functions. Particle [38] is a networking architecture that reduces network startup time by creating and optimizing network endpoints in burst-parallel serverless environments. In our method, we aim to consider the wide variety of function invocation behaviors and minimize the resource waste to reduce the cold start hit rate.

VII. Conclusion

This work presents a serverless framework to address two major issues in serverless computing. The proposed configuration algorithm utilizes both critical path and priority queue structures to automatically select and configure serverless functions in the workflow. Additionally, a gradient-based algorithm is designed to automatically start or terminate containers for upcoming executions. The experiments demonstrated that SSC achieved approximately 30% cost savings and reduced the cold start hit rate by almost 8%, which are beneficial for energy conservation and sustainable computing.

References

[1] E. Jonas et al., "Cloud programming simplified: A berkeley view on serverless computing," 2019, *arXiv: 1902.03383*.

[2] G. Adzic and R. Chatley, "Serverless computing: Economic and architectural impact," in *Proc. 11th joint Meeting Found. Softw. Eng.*, 2017, pp. 884–889.

[3] V. Ishakian, V. Muthusamy, and A. Slominski, "Serving deep learning models in a serverless platform," in *Proc. IEEE Int. Conf. Cloud Eng.*, 2018, pp. 257–262.

[4] S. Eismann et al., "A review of serverless use cases and their characteristics," 2020, *arXiv: 2008.11110*.

[5] Aws lambda. Accessed: Feb, 2023. [Online]. Available: https://aws.amazon.com/lambda/

[6] Google cloud functions. Accessed: Feb, 2023. [Online]. Available: https://cloud.google.com/functions/

[7] IBM cloud functions. Accessed: Feb, 2023. [Online]. Available: https://www.ibm.com/cloud/functions

[8] D. Rani and R. K. Ranjan, "A comparative study of SaaS, PaaS and IaaS in cloud computing," *Int. J. Adv. Res. Comput. Sci. Softw. Eng.*, vol. 4, no. 6, 2014.

[9] G. McGrath and P. R. Brenner, "Serverless computing: Design, implementation, and performance," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. Workshops*, 2017, pp. 405–410.

[10] P. Vahidinia, B. Farahani, and F. S. Aliee, "Mitigating cold start problem in serverless computing: A reinforcement learning approach," *IEEE Internet Things J.*, vol. 10, no. 5, pp. 3917–3927, Mar. 2023.

[11] H. Zhao et al., "faaShark: An end-to-end network traffic analysis system atop serverless computing platforms," *IEEE Trans. Netw. Sci. Eng.*, early access, Jul. 11, 2023, doi: 10.1109/TNSE.2023.3294406.

[12] M. Shahrad et al., "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX annu. tech. conf.*, 2020, pp. 205–218.

[13] Y. Zhang et al., "Faster and cheaper serverless computing on harvested resources," in *Proc. ACM SIGOPS 28th Symp. Operating Syst. Princ.*, 2021, pp. 724–739.

[14] G. A. S. Cassel, V. F. Rodrigues, R. da Rosa Righi, M. R. Bez, A. C. Nepomuceno, and C. A. da Costa, "Serverless computing for Internet of Things: A systematic literature review," *Future Gener. Comput. Syst.*, vol. 128, pp. 299–316, 2022.

[15] D. Pinto, J. P. Dias, and H. S. Ferreira, "Dynamic allocation of serverless functions in iot environments," in *Proc. IEEE 16th Int. Conf. Embedded Ubiquitous Comput.*, 2018, pp. 1–8.

[16] Y. Liu et al., "FuncPipe: A pipelined serverless framework for fast and cost-efficient training of deep learning models," in *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 6, no. 3, pp. 1–30, 2022.

[17] P. G. Sarroca and M. Sánchez-Artigas, "MLLess: Achieving cost efficiency in serverless machine learning training," *J. Parallel Distrib. Comput.*, 2023, Art. no. 104764.

[18] J. Jiang et al., "Towards demystifying serverless machine learning training," in *Proc. Int. Conf. Manage. Data*, 2021, pp. 857–871.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.

[19] H. Wang, D. Niu, and B. Li, "Distributed machine learning with a serverless architecture," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1288–1296.
[20] M. Yu, Z. Jiang, H. C. Ng, W. Wang, R. Chen, and B. Li, "Gillis: Serving large neural networks in serverless functions with automatic model partitioning," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 138–148.
[21] A. Ali, R. Pinciroli, F. Yan, and E. Smirni, "Batch: Machine learning inference serving on serverless platforms with adaptive batching," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2020, pp. 1–15.
[22] Y. Yang et al., "INFless: A native serverless system for low-latency, high-throughput inference," in *Proc. 27th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2022, pp. 768–781.
[23] K. Mahajan and R. Desai, "Serving distributed inference deep learning models in serverless computing," in *Proc. IEEE 15th Int. Conf. Cloud Comput.*, 2022, pp. 109–111.
[24] A. Mampage, S. Karunasekera, and R. Buyya, "Deadline-aware dynamic resource management in serverless computing environments," in *Proc. IEEE/ACM 21st Int. Symp. Cluster Cloud Internet Comput.*, 2021, pp. 483–492.
[25] M. Bilal, M. Canini, R. Fonseca, and R. Rodrigues, "With great freedom comes great opportunity: Rethinking resource allocation for serverless functions," in *Proc. 18th Eur. Conf. Comput. Syst.*, 2023, pp. 381–397.
[26] V. Gupta, S. Phade, T. Courtade, and K. Ramchandran, "Utility-based resource allocation and pricing for serverless computing," 2020, *arXiv: 2008.07793*.
[27] L. Schuler, S. Jamil, and N. Kühl, "AI-based resource allocation: Reinforcement learning for adaptive auto-scaling in serverless environments," in *Proc. IEEE/ACM 21st Int. Symp. Cluster Cloud Internet Comput.*, 2021, pp. 804–811.
[28] E. Hunhoff, S. Irshad, V. Thurimella, A. Tariq, and E. Rozner, "Proactive serverless function resource management," in *Proc. 6th Int. Workshop Serverless Comput.*, 2020, pp. 61–66.
[29] X. Li, P. Kang, J. Molone, W. Wang, and P. Lama, "KneeScale: Efficient resource scaling for serverless computing at the edge," in *Proc. IEEE 22nd Int. Symp. Cluster Cloud Internet Comput.*, 2022, pp. 180–189.
[30] N. Yuvaraj, T. Karthikeyan, and K. Praghash, "An improved task allocation scheme in serverless computing using gray wolf optimization (GWO) based reinforcement learning (RIL) approach," *Wirel. Pers. Commun.*, vol. 117, pp. 2403–2421, 2021.
[31] A. Pimpley et al., "Optimal resource allocation for serverless queries," 2021, *arXiv:2107.08594*.
[32] P. Silva, D. Fireman, and T. E. Pereira, "Prebaking functions to warm the serverless cold start," in *Proc. 21st Int. Middleware Conf.*, 2020, pp. 1–13.
[33] S. Agarwal, M. A. Rodriguez, and R. Buyya, "A reinforcement learning approach to reduce serverless function cold start frequency," in *Proc. 21st Int. Symp. Cluster Cloud Internet Comput.*, 2021, pp. 797–803.
[34] P.-M. Lin and A. Glikson, "Mitigating cold starts in serverless platforms: A pool-based approach," 2019, *arXiv: 1903.12221*.
[35] J. Shen, T. Yang, Y. Su, Y. Zhou, and M. R. Lyu, "Defuse: A dependency-guided function scheduler to mitigate cold starts on faas platforms," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 194–204.
[36] X. Liu et al., "FaaSLight: General application-level cold-start latency optimization for function-as-a-service in serverless computing," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, pp. 1–29, 2023.
[37] S. Lee, D. Yoon, S. Yeo, and S. Oh, "Mitigating cold start problem in serverless computing with function fusion," *Sensors*, vol. 21, no. 24, 2021, Art. no. 8416.
[38] S. Thomas, L. Ao, G. M. Voelker, and G. Porter, "Particle: Ephemeral endpoints for serverless networking," in *Proc. 11th ACM Symp. Cloud Comput.*, 2020, pp. 16–29.



Shanxing Pan is currently working toward the graduate degree in information security with Shanghai Jiao Tong University, China. His research interests are focused on resource schedule in cloud computing.



Hongyu Zhao is currently working toward the graduate degree in information security with Shanghai Jiao Tong University, China. His research interests are focused on resource schedule in cloud computing.



Zinuo Cai received the bachelor's degree in software engineering from Shanghai Jiao Tong University, China. He is currently working toward the graduate degree in computer science with Shanghai Jiao Tong University. His research interests are focused on resource schedule and system security in cloud computing.



Dongmei Li received the master's degree from the department of Materials Science, Beijing Institute of Technology. She is currently an senior engineer with the Department of Reliability at Beijing Institute of Microelectronics Technology. Her research interests include detection and failure analysis of integrated circuits.



Ruhui Ma (Member, IEEE) received the PhD degree in computer science from Shanghai Jiao Tong University. He is currently an associate professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research interests include cloud computing systems, AI systems, and machine learning.



Haibing Guan received the PhD degree from Tongji University in 1999. He is currently a professor with the School of Electronic Information and Electronic Engineering, Shanghai Jiao Tong University, and the director of the Shanghai Key Laboratory of Scalable Computing and Systems. His research interests include cloud/distributed computing and machine learning.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:02:28 UTC from IEEE Xplore. Restrictions apply.